In [1]:
```python
import os
from PIL import Image
import numpy as np
from sklearn.model_selection import train_test_split
import glob
import time

print(time.ctime())
#define four directorys/classes
Classification_Definition = ['birds','large_carnivores','small_mammals','ungulates']
image_directory = {}

for animal in Classification_Definition:
    # Define the directory where images are stored
    image_dir = 'C:/Users/msima/Desktop/data/' + animal
    # Get a list of all image files in the directory
    image_paths = glob.glob(os.path.join(image_dir, '*.jpg'))

    #collect file names in single array
    imagesFileLocation = []

    for img_path in image_paths:
        imagesFileLocation.append(img_path)

    image_directory[animal] = imagesFileLocation

for animal_species in image_directory:
        print("Animal:",animal_species,"has",len(image_directory[animal_species]), "pictures")

print(time.ctime())
```

```
Mon Apr  6 13:54:10 2026
Animal: birds has 258 pictures
Animal: large_carnivores has 5460 pictures
Animal: small_mammals has 1023 pictures
Animal: ungulates has 17673 pictures
Mon Apr  6 13:54:10 2026
```

In [2]:
```python
#using pillow, load each image, converting to 256x256 dimension
#convert to RGB channel
#print shapes of X and y to confirm sizes for CNN
```

```python
print(time.ctime())
images = []
animal_species_class = []
image_size = (224, 224) # Define a size for all images


for animal_species in image_directory:
    print("Processing:",animal_species)
    LiveCounter = 1
    for img_path in image_directory[animal_species]:
        if(LiveCounter % 500 == 0):
            print(animal_species," - LiveCounter:",LiveCounter)

        try:
            img = Image.open(img_path).convert('RGB') # Ensure image is in RGB format
            img = img.resize(image_size) # Resize the image
            images.append(np.array(img)) # Convert to NumPy array
            animal_species_class.append(np.array(animal_species)) # Convert to NumPy array
        except Exception as e:
            print(f"Error loading image {img_path}: {e}")

        LiveCounter += 1


# Convert the list of image arrays into a single NumPy array
X = np.array(images)
y = np.array(animal_species_class)

print(f"Shape of the image data array (X): {X.shape}")
print(f"Data type of the image data array: {X.dtype}")

print(f"Shape of the response data array (y): {y.shape}")
print(f"Data type of the response data array: {y.dtype}")

print(time.ctime())
```

```
Mon Apr  6 13:54:10 2026
Processing: birds
Processing: large_carnivores
large_carnivores  - LiveCounter: 500
large_carnivores  - LiveCounter: 1000
large_carnivores  - LiveCounter: 1500
large_carnivores  - LiveCounter: 2000
large_carnivores  - LiveCounter: 2500
large_carnivores  - LiveCounter: 3000
large_carnivores  - LiveCounter: 3500
large_carnivores  - LiveCounter: 4000
large_carnivores  - LiveCounter: 4500
large_carnivores  - LiveCounter: 5000
Processing: small_mammals
small_mammals  - LiveCounter: 500
small_mammals  - LiveCounter: 1000
Processing: ungulates
ungulates  - LiveCounter: 500
ungulates  - LiveCounter: 1000
ungulates  - LiveCounter: 1500
ungulates  - LiveCounter: 2000
ungulates  - LiveCounter: 2500
ungulates  - LiveCounter: 3000
ungulates  - LiveCounter: 3500
ungulates  - LiveCounter: 4000
ungulates  - LiveCounter: 4500
ungulates  - LiveCounter: 5000
ungulates  - LiveCounter: 5500
ungulates  - LiveCounter: 6000
ungulates  - LiveCounter: 6500
ungulates  - LiveCounter: 7000
ungulates  - LiveCounter: 7500
Error loading image C:/Users/msima/Desktop/data/ungulates\147963186.jpg: broken data stream when reading image file
ungulates  - LiveCounter: 8000
ungulates  - LiveCounter: 8500
ungulates  - LiveCounter: 9000
ungulates  - LiveCounter: 9500
ungulates  - LiveCounter: 10000
ungulates  - LiveCounter: 10500
ungulates  - LiveCounter: 11000
ungulates  - LiveCounter: 11500
ungulates  - LiveCounter: 12000
```

```
ungulates  - LiveCounter: 12500
ungulates  - LiveCounter: 13000
ungulates  - LiveCounter: 13500
ungulates  - LiveCounter: 14000
ungulates  - LiveCounter: 14500
ungulates  - LiveCounter: 15000
ungulates  - LiveCounter: 15500
ungulates  - LiveCounter: 16000
ungulates  - LiveCounter: 16500
ungulates  - LiveCounter: 17000
ungulates  - LiveCounter: 17500
Shape of the image data array (X): (24413, 224, 224, 3)
Data type of the image data array: uint8
Shape of the response data array (y): (24413,)
Data type of the response data array: <U16
Mon Apr  6 14:30:58 2026
```

In [3]:

```
print(time.ctime())
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.15, stratify=y, random_state=42)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

print("Data preparation complete! You now have X_train, X_test, y_train, and y_test ready for your CNN.")
print(time.ctime())
```

```
Mon Apr  6 14:30:58 2026
Shape of X_train: (20751, 224, 224, 3)
Shape of X_test: (3662, 224, 224, 3)
Shape of y_train: (20751,)
Shape of y_test: (3662,)
Data preparation complete! You now have X_train, X_test, y_train, and y_test ready for your CNN.
Mon Apr  6 14:31:05 2026
```

In [4]:

```
#create validation data
print(time.ctime())
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.15, stratify=y_train, random_state=42

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_val: {X_val.shape}")
```

```
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_val: {y_val.shape}")

print("Data preparation complete! You now have X_train, X_val, y_train, and y_val ready for your CNN.")
print(time.ctime())
```

```
Mon Apr  6 14:31:05 2026
Shape of X_train: (17638, 224, 224, 3)
Shape of X_val: (3113, 224, 224, 3)
Shape of y_train: (17638,)
Shape of y_val: (3113,)
Data preparation complete! You now have X_train, X_val, y_train, and y_val ready for your CNN.
Mon Apr  6 14:31:30 2026
```

In [5]:

```
import time
import tensorflow as tf
from tensorflow.keras import layers, models

print(time.ctime())

# Define the CNN model using the updated Input layer style
model = models.Sequential([
    layers.Input(shape=X_train.shape[1:]), # Define input shape here
    layers.Conv2D(32, (3, 3), activation='relu'), # Remove input_shape from here
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(len(Classification_Definition), activation='softmax')
])

# Display the model summary
model.summary()
print(time.ctime())
```

```
Mon Apr  6 14:31:37 2026
```

**Model: "sequential"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 222, 222, 32) | 896 |
| max_pooling2d (MaxPooling2D) | (None, 111, 111, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 109, 109, 64) | 18,496 |
| max_pooling2d_1 (MaxPooling2D) | (None, 54, 54, 64) | 0 |
| conv2d_2 (Conv2D) | (None, 52, 52, 64) | 36,928 |
| flatten (Flatten) | (None, 173056) | 0 |
| dense (Dense) | (None, 64) | 11,075,648 |
| dense_1 (Dense) | (None, 4) | 260 |

**Total params:** 11,132,228 (42.47 MB)

**Trainable params:** 11,132,228 (42.47 MB)

**Non-trainable params:** 0 (0.00 B)

```
Mon Apr  6 14:31:37 2026
```

In [6]:
```
print(time.ctime())

# Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

print(time.ctime())
```

```
Mon Apr  6 14:31:37 2026
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuD
NN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Mon Apr  6 14:31:37 2026
Mon Apr  6 14:31:37 2026
```

In [7]:
```
print(time.ctime())
from sklearn.preprocessing import LabelEncoder
```

```python
# Initialize LabelEncoder
label_encoder = LabelEncoder()

# Fit on the entire 'y' array to ensure all unique labels are encoded consistently
# and then transform y_train and y_test
# We're transforming y (all original labels) to get the full mapping, then transforming y_train and y_test separately
# Alternatively, fit on y_train and transform both y_train and y_test

# First, combine y_train and y_test to fit the encoder across all possible labels
# This ensures that the same integer is assigned to the same class across both sets
y_combined = np.concatenate((y_train, y_val, y_test), axis=0)
label_encoder.fit(y_combined)

y_train_encoded = label_encoder.transform(y_train)
y_val_encoded = label_encoder.transform(y_val)
y_test_encoded = label_encoder.transform(y_test)

# Print the first few encoded labels and original labels to verify
print(f"Original y_train labels (first 5): {y_train[:5]}")
print(f"Encoded y_train labels (first 5): {y_train_encoded[:5]}")
print(f"Original y_val labels (first 5): {y_val[:5]}")
print(f"Encoded y_val labels (first 5): {y_val_encoded[:5]}")
print(f"Original y_test labels (first 5): {y_test[:5]}")
print(f"Encoded y_test labels (first 5): {y_test_encoded[:5]}")

print(f"Unique classes and their encoded values: {list(label_encoder.classes_)} -> {label_encoder.transform(label_enc

# Now, reassign y_train and y_test to their encoded versions
y_train = y_train_encoded
y_val = y_val_encoded
y_test = y_test_encoded

print(f"Shape of encoded y_train: {y_train.shape}")
print(f"Shape of encoded y_test: {y_test.shape}")

print(time.ctime())
```

```
Mon Apr  6 14:31:37 2026
Original y_train labels (first 5): ['large_carnivores' 'ungulates' 'ungulates' 'ungulates' 'ungulates']
Encoded y_train labels (first 5): [1 3 3 3 3]
Original y_val labels (first 5): ['ungulates' 'ungulates' 'ungulates' 'large_carnivores' 'ungulates']
Encoded y_val labels (first 5): [3 3 3 1 3]
Original y_test labels (first 5): ['ungulates' 'large_carnivores' 'ungulates' 'large_carnivores' 'ungulates']
Encoded y_test labels (first 5): [3 1 3 1 3]
Unique classes and their encoded values: [np.str_('birds'), np.str_('large_carnivores'), np.str_('small_mammals'), n
p.str_('ungulates')] -> [0 1 2 3]
Shape of encoded y_train: (17638,)
Shape of encoded y_test: (3662,)
Mon Apr  6 14:31:37 2026
```

In [8]:

```
print(time.ctime())
# Define Early Stopping callback
early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',  # Monitor validation loss
    patience=5,          # Number of epochs with no improvement after which training will be stopped
    restore_best_weights=True  # Restore model weights from the epoch with the best value of the monitored quantity
)

# Train the model with early stopping
history = model.fit(X_train, y_train, epochs=100, # Increased epochs as early stopping will handle stopping
                    validation_data=(X_val, y_val),
                    callbacks=[early_stopping])

print(time.ctime())
```

```
Mon Apr  6 14:31:37 2026
Epoch 1/100
552/552 ━━━━━━━━━━━━━━━━━━━━ 134s 240ms/step - accuracy: 0.7715 - loss: 4.5853 - val_accuracy: 0.8673 - val_loss: 0.3
782
Epoch 2/100
552/552 ━━━━━━━━━━━━━━━━━━━━ 131s 237ms/step - accuracy: 0.8777 - loss: 0.3520 - val_accuracy: 0.9293 - val_loss: 0.2
254
Epoch 3/100
552/552 ━━━━━━━━━━━━━━━━━━━━ 128s 231ms/step - accuracy: 0.9438 - loss: 0.1736 - val_accuracy: 0.9313 - val_loss: 0.2
297
Epoch 4/100
552/552 ━━━━━━━━━━━━━━━━━━━━ 130s 236ms/step - accuracy: 0.9715 - loss: 0.0940 - val_accuracy: 0.9489 - val_loss: 0.1
765
Epoch 5/100
552/552 ━━━━━━━━━━━━━━━━━━━━ 131s 238ms/step - accuracy: 0.9816 - loss: 0.0587 - val_accuracy: 0.9464 - val_loss: 0.2
208
Epoch 6/100
552/552 ━━━━━━━━━━━━━━━━━━━━ 130s 236ms/step - accuracy: 0.9794 - loss: 0.0708 - val_accuracy: 0.9547 - val_loss: 0.2
224
Epoch 7/100
552/552 ━━━━━━━━━━━━━━━━━━━━ 131s 237ms/step - accuracy: 0.9866 - loss: 0.0504 - val_accuracy: 0.9531 - val_loss: 0.2
331
Epoch 8/100
552/552 ━━━━━━━━━━━━━━━━━━━━ 133s 242ms/step - accuracy: 0.9884 - loss: 0.0394 - val_accuracy: 0.9627 - val_loss: 0.2
194
Epoch 9/100
552/552 ━━━━━━━━━━━━━━━━━━━━ 131s 237ms/step - accuracy: 0.9868 - loss: 0.0480 - val_accuracy: 0.9611 - val_loss: 0.2
657
Mon Apr  6 14:51:18 2026
```

In [9]:

```python
from sklearn.metrics import classification_report

print(time.ctime())

# Evaluate the model for Loss and Accuracy
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Test Loss: {loss:.4f}")
print(f"Test Accuracy: {accuracy:.4f}")

# Generate detailed metrics
y_pred_probs = model.predict(X_test)
y_pred = np.argmax(y_pred_probs, axis=1)
```

```python
# Print Precision, Recall, and F1-Score for each class
print("\nDetailed Classification Report:")
print(classification_report(y_test, y_pred, target_names=label_encoder.classes_))

print(time.ctime())
```

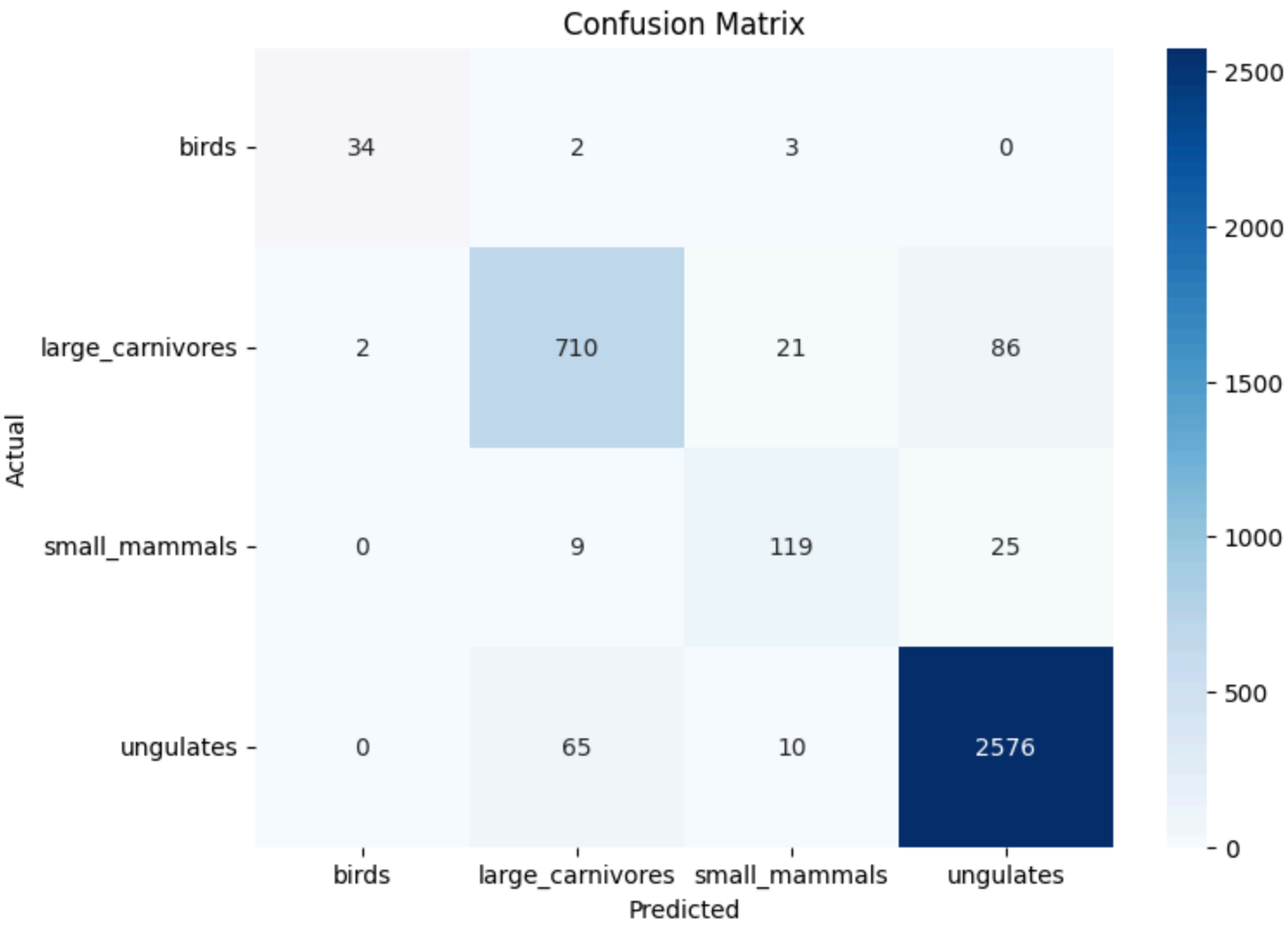
```
Mon Apr  6 14:51:18 2026
115/115 ━━━━━━━━━━━━━━━━━━━━ 8s 66ms/step - accuracy: 0.9391 - loss: 0.2116
Test Loss: 0.2116
Test Accuracy: 0.9391
115/115 ━━━━━━━━━━━━━━━━━━━━ 7s 62ms/step

Detailed Classification Report:
                  precision    recall  f1-score   support

           birds       0.94      0.87      0.91        39
large_carnivores       0.90      0.87      0.88       819
   small_mammals       0.78      0.78      0.78       153
       ungulates       0.96      0.97      0.97      2651

        accuracy                           0.94      3662
       macro avg       0.90      0.87      0.88      3662
    weighted avg       0.94      0.94      0.94      3662

Mon Apr  6 14:51:34 2026
```

In [10]:

```python
import numpy as np
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Make predictions on the test set
y_pred_probs = model.predict(X_test)
y_pred = np.argmax(y_pred_probs, axis=1)

# Create the confusion matrix
cm = confusion_matrix(y_test, y_pred)

# Print the confusion matrix
print("Confusion Matrix:")
print(cm)
```

```python
# Visualize the confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.yticks(rotation=0)
Filename = 'C:/Users/msima/Desktop/Results/confusion_matrix_values.png'
plt.savefig(Filename)
plt.show()
```

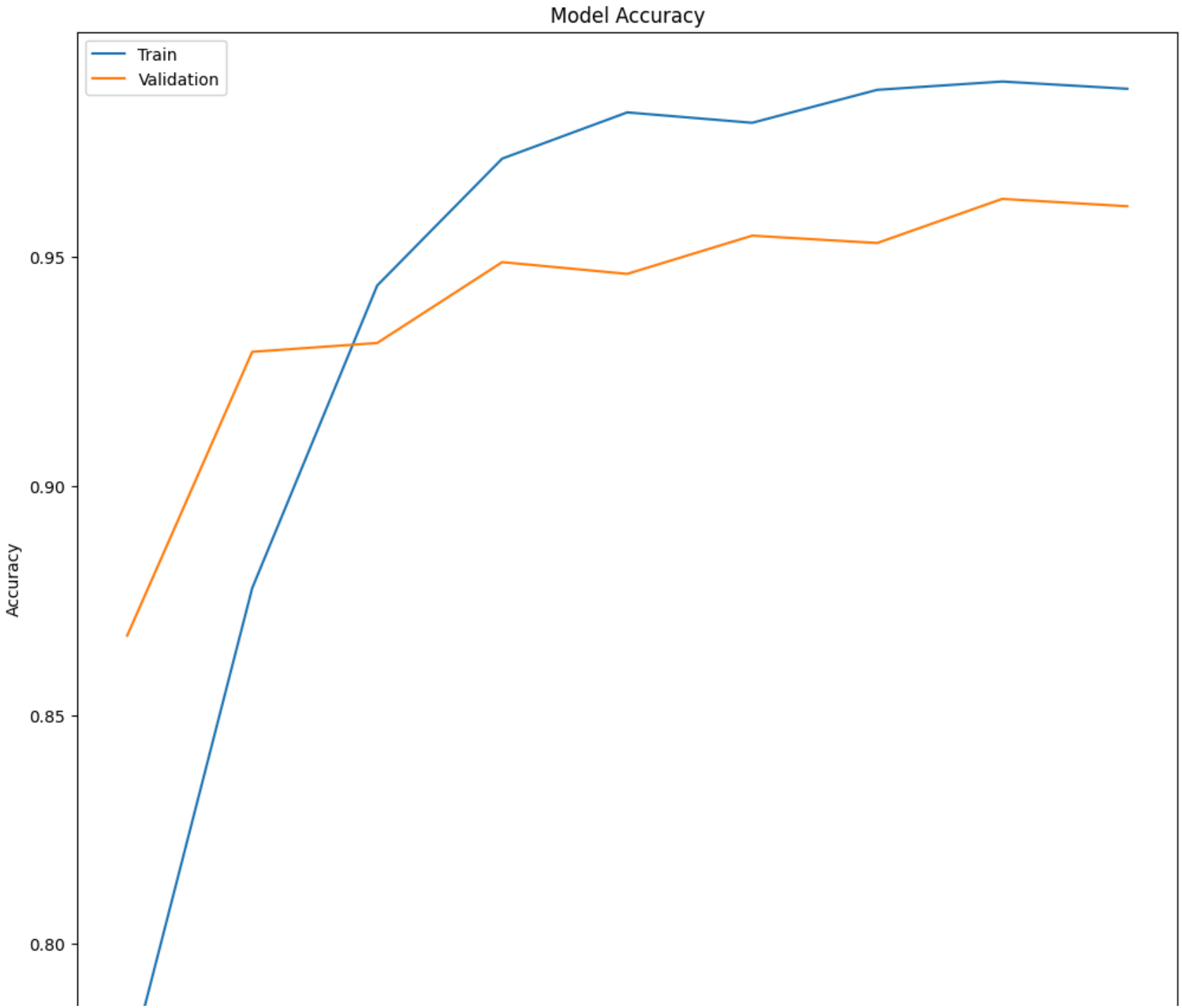
```
115/115 ━━━━━━━━━━━━━━━━━━━━ 7s 61ms/step
Confusion Matrix:
[[  34    2    3    0]
 [   2  710   21   86]
 [   0    9  119   25]
 [   0   65   10 2576]]
```

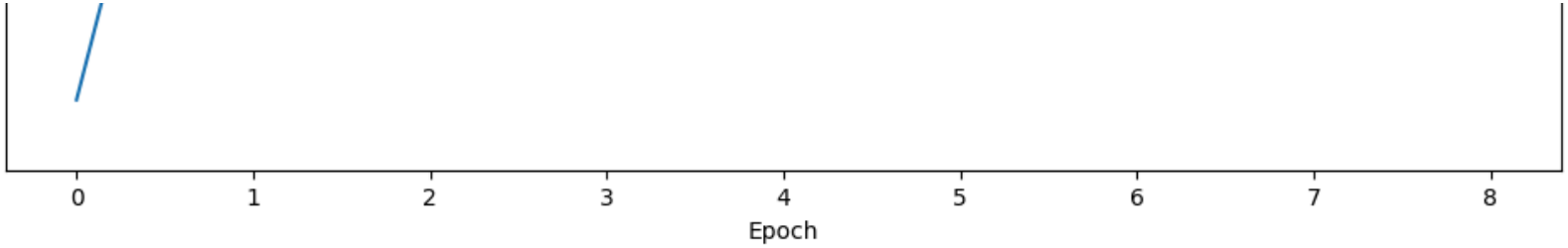



In [ ]:

In [11]:
```
import matplotlib.pyplot as plt

# Plot training history
plt.figure(figsize=(12,12))

# Plot training & validation accuracy values
```

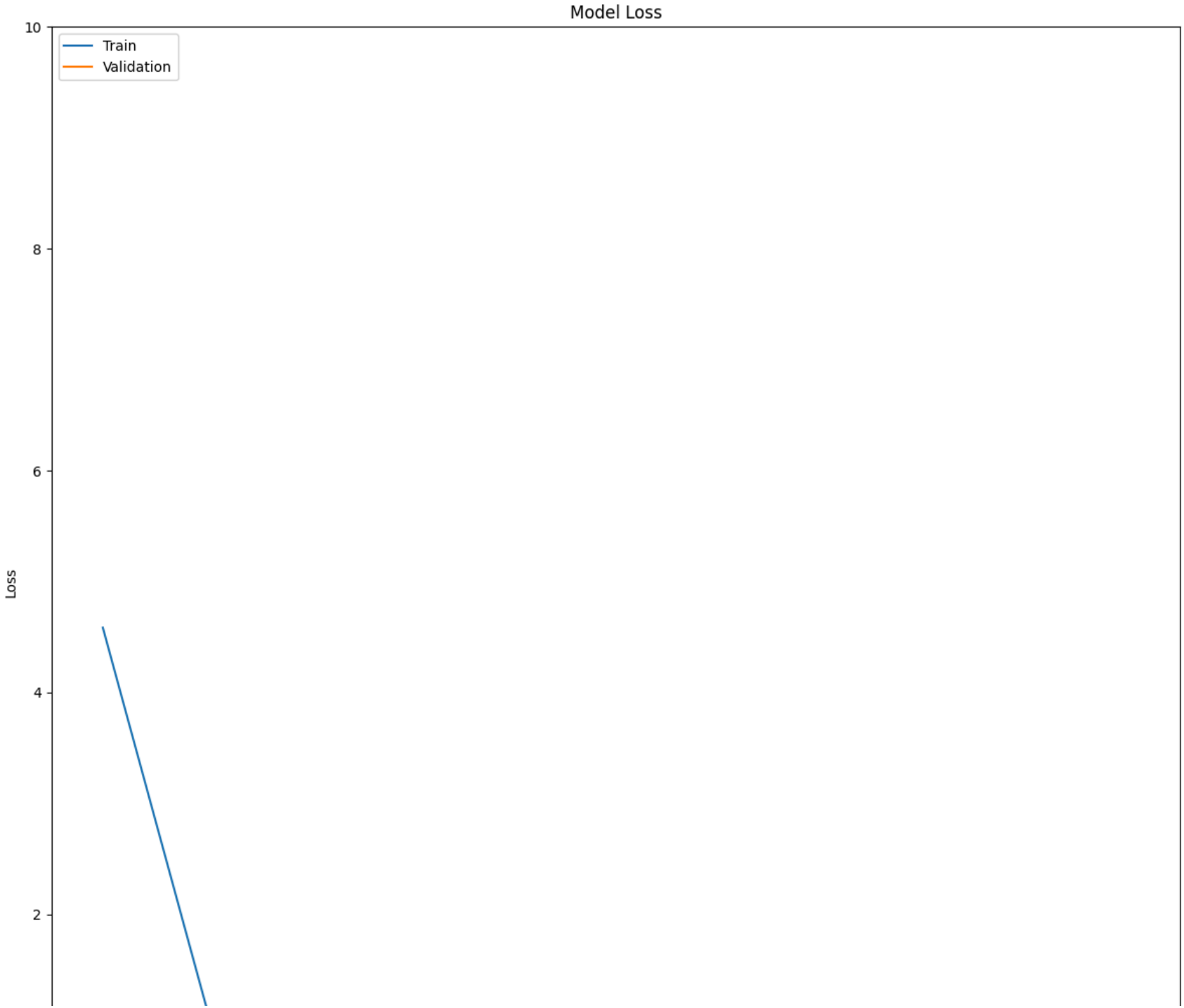
```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
Filename = 'C:/Users/msima/Desktop/Results/Line_charts_model_accuracy.png'
plt.savefig(Filename)

plt.show()
```

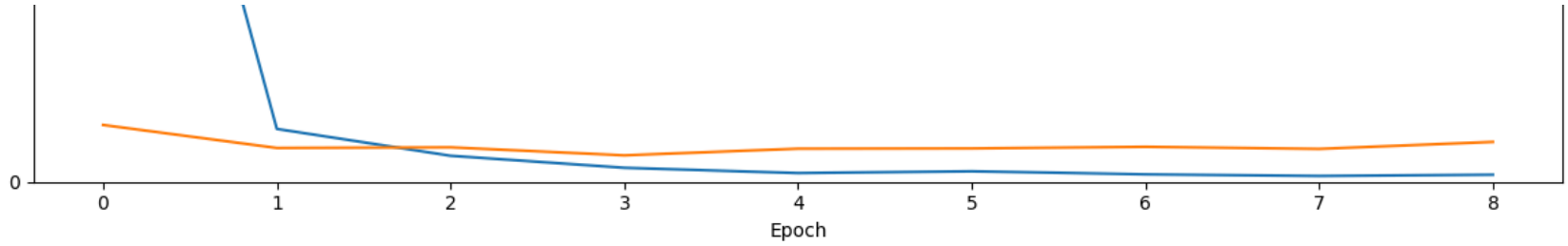

Model Accuracy
Train
Validation
Accuracy
0.95
0.90
0.85
0.80



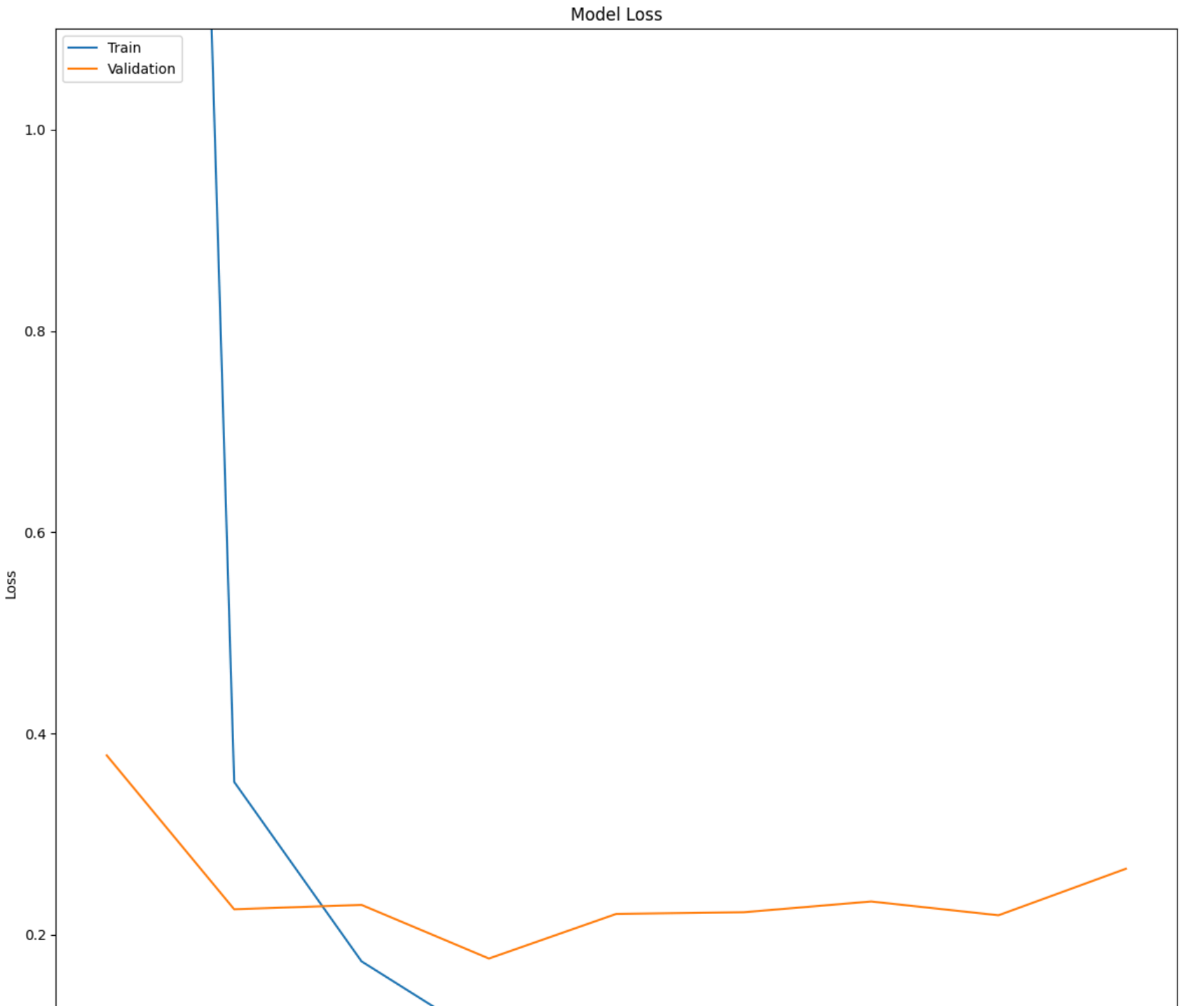

In [12]:

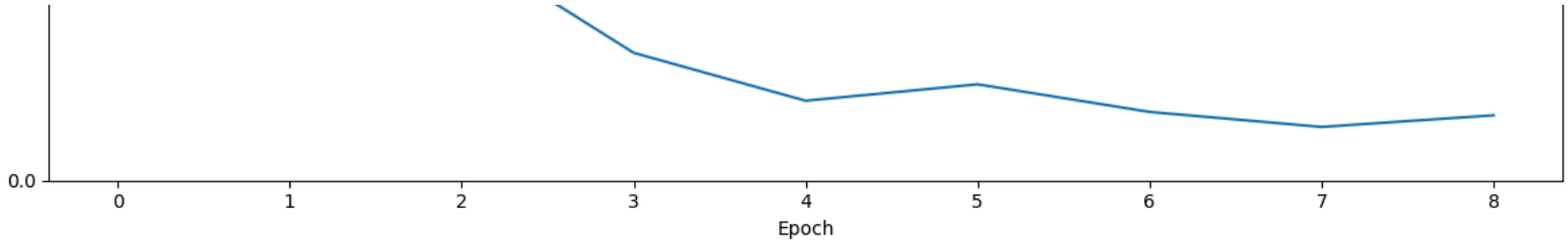
```
import matplotlib.pyplot as plt

# Plot training history
plt.figure(figsize=(12, 12))

# Plot training & validation loss values
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylim(0,10)
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.tight_layout()
Filename = 'C:/Users/msima/Desktop/Results/Line_charts_model_loss.png'
plt.savefig(Filename)

plt.show()
```


Model Loss
Train
Validation
Loss
10
8
6
4
2




In [13]:
```
import matplotlib.pyplot as plt

# Plot training history
plt.figure(figsize=(12, 12))

# Plot training & validation loss values
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylim(0,1.1)
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.yticks(rotation=0)
plt.tight_layout()
Filename = 'C:/Users/msima/Desktop/Results/Line_charts_model_loss_zoomed_in.png'
plt.savefig(Filename)

plt.show()
```


Model Loss
Train
Validation
Loss
1.0
0.8
0.6
0.4
0.2

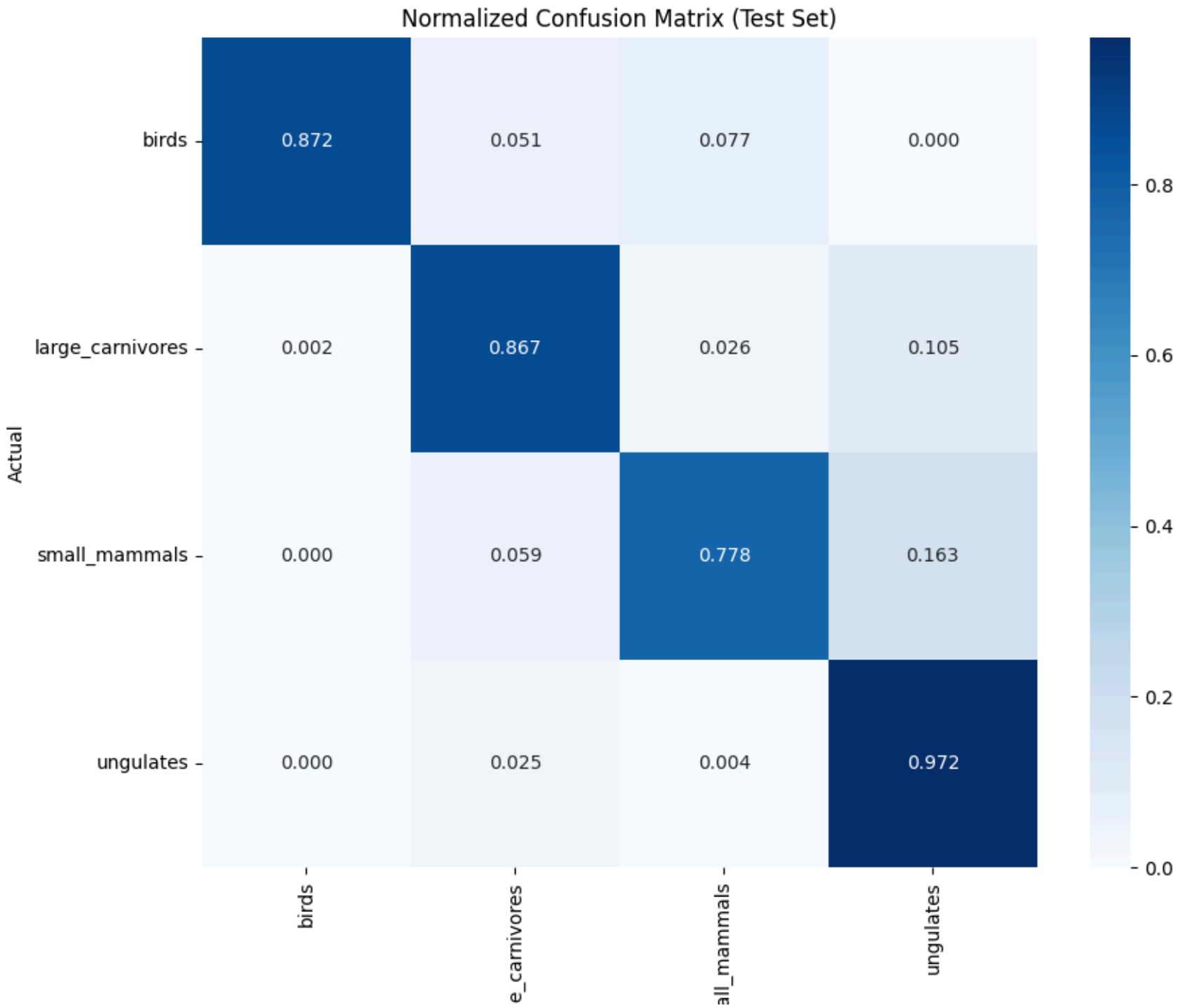



In [14]:

```
import numpy as np
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Make predictions on the test set
y_pred_probs = model.predict(X_test)
y_pred = np.argmax(y_pred_probs, axis=1)

# Create the confusion matrix
cm = confusion_matrix(y_test, y_pred, normalize='true')

# Normalize the confusion matrix to show percentages
cm_normalized = cm.astype('float') / cm.sum(axis=1, keepdims=True)

# Print the confusion matrix (raw counts for reference)
print("Confusion Matrix (Raw Counts):")
print(cm)

# Visualize the confusion matrix with percentages
plt.figure(figsize=(10, 8))
sns.heatmap(cm_normalized, annot=True, fmt='.3f', cmap='Blues',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.xticks(rotation=90)
plt.yticks(rotation=0)

plt.yticks(rotation=0)
plt.title('Normalized Confusion Matrix (Test Set)')
Filename = 'C:/Users/msima/Desktop/Results/confusion_matrix_percent.png'
plt.savefig(Filename)
```

```python
plt.show()
print(time.ctime())
```

```
115/115 ━━━━━━━━━━━━━━━━━━━━ 7s 62ms/step
Confusion Matrix (Raw Counts):
[[0.87179487 0.05128205 0.07692308 0.        ]
 [0.002442   0.86691087 0.02564103 0.10500611]
 [0.         0.05882353 0.77777778 0.16339869]
 [0.         0.02451905 0.00377216 0.97170879]]
```


Normalized Confusion Matrix (Test Set)
Actual
birds
large_carnivores
small_mammals
ungulates
0.872
0.051
0.077
0.000
0.002
0.867
0.026
0.105
0.000
0.059
0.778
0.163
0.000
0.025
0.004
0.972
birds
e_carnivores
all_mammals
ungulates
0.8
0.6
0.4
0.2
0.0

```
Mon Apr  6 14:51:50 2026
```